

AI 智能对话系统

前端实习项目简历描述 — Vue 3 + Node.js + WebSocket

Vue 3 Composition API

Pinia 状态管理

Socket.io

Vite 5

Element Plus

DeepSeek API

JWT 认证

MySQL

一、项目简介

基于 Vue 3 + Pinia + Vite 开发的 AI 对话单页应用，集成 Socket.io 实时流式通信与 DeepSeek API；实现了 JWT 用户认证、多会话管理、打字机效果实时渲染、多轮对话上下文感知等功能。采用前后端分离架构（Vue 3 ↔ Express.js ↔ MySQL），具备完整的现代化工程化开发流程。

二、技术栈

类别	技术	用途
前端框架	Vue 3 (Composition API, <script setup>)	组件化 SPA
构建工具	Vite 5	毫秒级 HMR，生产构建
状态管理	Pinia 2	模块化全局状态管理
路由	Vue Router 4	导航守卫，登录态拦截
UI 组件库	Element Plus 2	表单、消息提示、图标系统

HTTP 客户端	Axios	请求/响应拦截器链路
实时通信	Socket.io Client	WebSocket 流式传输
后端框架	Express.js + Node.js	RESTful API
数据库	MySQL + mysql2/promise	连接池，异步操作
认证	JWT + bcryptjs	Token 鉴权，密码加密
AI 集成	DeepSeek API (SSE Stream)	流式 AI 对话

三、项目亮点

亮点一：基于 Vue 3 Composition API 构建高复用组件体系

采用 **Vue 3 <script setup>** 组合式 API 组织业务逻辑，将聊天界面解构为四个职责单一的核心组件：

- **Sidebar.vue** — 对话列表面板，支持新建、切换、删除操作
- **ChatMessage.vue** — 消息气泡渲染，通过 props 区分用户/AI 角色样式
- **ChatInput.vue** — 智能输入框，支持 Enter 发送 / Shift+Enter 换行

通过 **props 向下传参**、**emit 向上通信** 实现组件间解耦。Pinia 模块化设计将全局状态拆分为 `userStore`（认证态）与 `chatStore`（对话态），支持多会话并发切换时状态隔离，避免数据污染。

✨ **技术亮点：** 组件粒度划分合理，每个组件聚焦单一职责；Pinia store 按业务域拆分，`currentConversationId` 字段隔离不同会话的消息列表，从根本上杜绝数据串扰。

亮点二：基于 Socket.io 实现流式 AI 对话的实时渲染

前端接入 Socket.io 客户端与服务端建立 WebSocket 长连接，实现 **逐字推流** → **即时渲染** 的低延迟链路：

1. 用户发送消息 → Socket emit `chat-message`
2. 服务端调用 DeepSeek API (`stream: true`) → 解析 SSE 数据块
3. 服务端实时转发 `chat-chunk` 事件 → 客户端逐块追加文本
4. 流式结束 → 服务端保存完整回复 → emit `chat-complete`

利用 **Vue 响应式系统**，每次收到 `chat-chunk` 仅追加至当前最后一个 assistant 消息的 `content` 字段，Virtual DOM diff 仅 patch 对应 DOM 节点，避免全列表重渲染。配合 **CSS keyframe 弹跳点动画** 在流式传输期间提供清晰的加载状态反馈，显著降低用户等待焦虑。

✨ **技术亮点：** 相比 HTTP 轮询方案，WebSocket 全双工通信延迟从秒级降至毫秒级；Vue 响应式追踪 + Virtual DOM diff 确保流式渲染性能；自动重连机制增强连接稳定性。

亮点三：完整的 JWT 认证体系与请求拦截链路

实现了 **登录注册** → **JWT 签发** → **本地持久化** → **自动携带** → **过期处理** 的完整闭环：

- 用户注册/登录后，服务端签发 7 天有效期的 JWT，返回前端存储至 `localStorage`
- **Axios 请求拦截器** 自动从 `localStorage` 读取 Token 并注入 `Authorization` 请求头，开发者无需手动传参
- **Axios 响应拦截器** 统一捕获 401 状态码，自动清除失效凭证并重定向至登录页
- **Vue Router 全局导航守卫** 双重保障，未登录用户无法访问聊天页面

```
// 请求拦截器 - 自动携带 Token
http.interceptors.request.use(config => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

```

    return config
  })

  // 响应拦截器 - 统一 401 处理
  http.interceptors.response.use(
    res => res.data,
    error => {
      if (error.response?.status === 401) {
        localStorage.clear(); router.push('/login')
      }
      return Promise.reject(error)
    }
  )
)

```

✨ **技术亮点：**拦截器模式将横切关注点（认证）与业务逻辑分离；前端 + 路由双重认证校验，防御纵深；Token 过期自动清理，避免无效请求。

亮点四：工程化构建与开发体验优化

基于 **Vite 5** 搭建现代化前端工程化环境：

- 利用 ESM 原生模块加载能力，实现 **毫秒级热更新**，大幅提升开发迭代效率
- 配置开发代理转发 `/api` 请求至后端，解决开发环境跨域问题
- 集成 **Element Plus** 组件库，结合 CSS 自定义属性覆盖实现 **紫色渐变品牌主题**
- 前后端完全分离，通过环境变量管理不同环境（开发/生产）的 API 地址

```

// vite.config.js
export default defineConfig({
  plugins: [vue()],
  server: {
    proxy: { '/api': 'http://localhost:3000' }
  },
  resolve: {
    alias: { '@': fileURLToPath(new URL('./src', import.meta.url)) }
  }
})

```

✨ **技术亮点:** Vite 的 ESM 原生 HMR 相比 Webpack 的 bundle HMR, 大型项目中速度提升 5-10 倍; 路径别名 (@/) 规范导入路径; 代理配置隔离前后端开发环境。

亮点五: AI 大模型工程化落地实践

对接 DeepSeek API, 将 AI 能力完整融入应用业务逻辑:

- **流式 SSE 解析:** 使用 `responseType: 'stream'` 接收 Server-Sent Events, 逐行解析 `data:` 前缀数据块, 提取增量内容
- **多轮对话上下文管理:** 每次用户发送消息时, 从数据库取出该会话的完整历史记录, 拼接为 `{ role, content }` 格式传入 API, 保持对话语义连贯
- **自动标题生成:** 首轮 AI 回复完成后, 自动截取用户首条消息前 30 个字符作为对话标题, 提升会话管理效率

```
async *chatStream(messages) {
  const response = await axios.post(DEEPSEEK_API_URL, {
    model: 'deepseek-chat',
    messages,
    stream: true
  }, { responseType: 'stream' })

  // 逐行解析 SSE 数据块, yield 每个 content 片段
  for await (const chunk of parseSSE(response.data)) {
    if (chunk) yield chunk
  }
}
```

✨ **技术亮点:** Async Generator 模式优雅处理流式数据消费; 完整对话历史拼接保证多轮交互体验; SSE 解析兼容多种流式 API 格式, 具备可迁移性。

亮点六: 全栈架构视野与数据库设计能力

采用 前后端完全分离架构, 完整数据链路为:

独立设计关系型数据库模型，三张数据表通过外键关联：

表名	核心字段	说明
users	id, username, email, password_hash	用户账户, password_hash 不返回前端
conversations	id, user_id (FK), title	会话归属用户, 级联删除
messages	id, conversation_id (FK), role, content	消息归属会话, role 枚举 user/assistant

- 使用 mysql2/promise 连接池管理数据库连接，提升并发性能
- 外键约束搭配 ON DELETE CASCADE 保证数据一致性，删除用户/会话时自动清理关联数据

🌟 技术亮点：展示了对全栈开发流程的完整理解，具备独立设计数据模型和与后端团队高效协作的能力。

四、面试追问应答准备

Q1：为什么选择 Socket.io 而不是 HTTP 轮询？

WebSocket 全双工通信在流式场景下具有天然优势——每个 token 到达服务端即可立即推送至客户端，端到端延迟低至毫秒级。而 HTTP 轮询存在固定间隔（通常 1-5 秒），且每秒数百次空请求对服务端造成不必要的资源浪费。Socket.io 还提供自动重连、房间隔离等高级特性，非常适合对话类实时应用。

Q2：流式渲染如何保证性能和正确性？

Vue 的响应式系统精确追踪每个 `messages` 数组中的对象变化。每次收到 `chat-chunk` 时，仅更新最后一个 assistant 消息的 `content` 属性，Virtual DOM diff 仅 patch 对应 DOM 节点，不会全量渲染。配合 `key="message.id"` 保证列表 diff 准确，`nextTick` + `scrollToBottom` 确保新内容可见。

Q3：如何防范 XSS 攻击？

用户输入和 AI 回复默认使用 `v-text` 指令进行 HTML 转义，避免恶意脚本注入。AI 内容中的换行符通过 `replace(/\n/g, '<br>')` 安全转换，而非直接使用 `v-html` 渲染不可信内容。虽然本项目 AI 数据源相对可信，但作为安全编码习惯始终保持输入输出的转义处理。

Q4：多会话切换时如何保证消息不串？

Pinia store 中通过 `currentConversationId` 字段隔离不同会话的消息列表。切换会话时：① 通过 API 获取该会话的完整消息历史；② 将 `messages` 数组整体替换为新数据；③ 通过 Socket emit `join-conversation` 加入对应房间。同一时刻 `messages` 仅维护一个会话的数据，从根本上杜绝数据串扰。

Q5：项目中遇到的挑战及解决思路？

最大的挑战是流式数据的渲染与状态管理。最初尝试在组件内部直接维护 Socket 连接，但组件卸载后连接丢失。解决方案：将 Socket.io 连接提升至 Pinia store 中管理，通过 `initSocket()` 在 ChatView 挂载时统一初始化，结合 `onUnmounted` 清理监听器，确保连接生命周期与视图同步。同时使用 `isTyping` 标志位管理 UI 状态与用户输入的互斥关系，避免流式传输期间重复发送。